How my finger lock's neural network works

Voice-over script · ME193 · Fiona Gettelman · movie length 4:46

How to record

1. Watch **vault_movie_guide.mp4** once. A computer voice reads this script at the right pace, with captions.

2. Open **vault_movie.mp4** (silent) in iMovie, choose *Record Voiceover*, and read the script while it plays. Each scene below shows the time it starts in the movie.

3. Don't rush. Every scene has a short pause at the start and end. If you're a little slow or fast, just start the next scene when its start time comes up.

4. Export, and the movie is done.

Scenes

Starts	Scene	Length
0:00	Title (no voice-over)	4 s
0:04	1. The safe	15 s
0:19	2. What the computer sees	23 s
0:42	3. One neuron	16 s
0:58	4. Convolution	30 s
1:28	5. Width	21 s
1:50	6. Max pooling	21 s
2:11	7. Depth	28 s
2:39	8. The answer	20 s
2:59	9. Learning from mistakes: backpropagation	21 s
3:21	10. Gradient descent	19 s
3:40	11. The data	20 s
4:01	12. Something new: Grad-CAM	24 s
4:25	13. On the Arduino	17 s
4:42	End card (no voice-over)	4 s

1. The safe 0:04 – 0:19

On screen: The safe: a hand presses the button, the LEDs go white, it shows 1, 4, 2 and the safe opens.

This is my safe.

There's no keypad.

You press the button, show the camera three numbers with your fingers, and if they're right, the servo swings it open.

The thing reading your fingers is a neural network I built from scratch.

Here's how it works.

2. What the computer sees 0:19 – 0:42

On screen: A photo of my hand gets a 64 × 64 grid; a magnifier shows the red, green and blue numbers of 16 pixels.

The camera sees a picture, but a computer only sees numbers.

I crop the middle of the frame and shrink it to 64 by 64 pixels.

Each pixel has a red, a green and a blue value, so every picture is 12,288 numbers.

The network's job is to turn those numbers into one answer: zero, one, two, three, four or five fingers.

3. One neuron 0:42 – 0:58

On screen: Three inputs times their weights add up in a neuron (1.01), then the ReLU graph: -1.3 becomes 0, 1.01 goes through.

The building block is a neuron.

It multiplies each input by a weight, adds them all up, and passes the total through ReLU: negative becomes zero, positive goes straight through.

That one bend is what lets a stack of these learn shapes, not just straight lines.

4. Convolution 0:58 – 1:28

On screen: A 3 × 3 filter slides over a 10 × 10 picture and builds an edge map; real filters from my network; 448 vs ≈ 200,000.

For pictures I use a convolution.

Instead of one neuron looking at the whole image, a tiny 3 by 3 filter, just nine weights, slides across the picture one step at a time.

At every spot it multiplies and adds, and writes one number onto a new map.

If the filter has learned to find an edge, the map lights up wherever there's an edge, anywhere in the picture, using the same nine weights.

That sharing is why my first layer needs only 448 weights instead of almost 200,000.

5. Width

1:28 – 1:50

On screen: 16 real feature maps fan out, then stacks of 16, 32, 64 and 128 maps.

One filter finds one pattern, so each layer has many of them.

That's the width.

My first layer has 16 filters, so it makes 16 feature maps: edges at different angles, light against dark, skin against the background.

Each layer after that doubles it: 32, then 64, then 128.

6. Max pooling

1:50 – 2:11

On screen: A blue 2 × 2 frame keeps the biggest number of each block; maps shrink 64 → 32 → 16 → 8; a moved finger, same answer.

After each convolution comes max pooling.

I look at every 2 by 2 block and keep only the biggest number.

The map shrinks to half its width and height, 64, then 32, 16 and 8, so there's far less to compute.

And because it keeps the strongest signal, a finger that moves over by one pixel gives almost the same answer.

7. Depth

2:11 – 2:39

On screen: Blocks conv 1 to conv 4 stack up while circles on the photo grow (3, 8, 18, 38 pixels); then the bar chart of depths.

Stacking layers is the depth.

My network has four convolution layers, and each one looks at the one below it, so it sees a bigger patch of the original picture: 3 pixels, then 8, then 18, then 38, enough to see a whole hand.

Edges become bits of finger, bits become fingers, and fingers become "three in a row".

In my tests, depth mattered most: with only two layers, the network couldn't see a whole hand and got less than half right.

8. The answer

2:39 – 2:59

On screen: 128 feature maps squash into beads that pour into cups 0 to 5; cup 3 fills: 100%. Stamp: 98,694 weights.

At the end, global average pooling squashes each of the 128 maps down to one number, and a last layer turns those into six scores, one for each count.

Softmax turns the scores into probabilities, and the biggest one is the answer.

Altogether, the network has 98,694 weights.

9. Learning from mistakes: backpropagation

2:59 – 3:21

On screen: Picture → conv blocks → cups; random weights guess wrong; a red backpropagation arrow sweeps back and turns the dials.

At first the weights are random, so the answers are garbage.

Training fixes that.

I show it a picture and the right answer, and the loss measures how wrong it was.

Backpropagation then works backwards through every layer using the chain rule, and figures out, for every single weight: if I nudge you a little, does the loss go up or down, and by how much?

10. Gradient descent

3:21 – 3:40

On screen: A marble rolls down the loss hill one step at a time, overshoots a bump, and settles in the valley.

Gradient descent uses that answer to move every weight a small step downhill.

Do that thousands of times, on batches of 64 pictures, and the loss rolls down into a valley where the answers are right.

I wrote that step myself, with momentum, so it keeps rolling through the small bumps instead of getting stuck.

11. The data

3:40 – 4:01

On screen: Photos of my hand pile up; one gets turned, zoomed, mirrored, darkened; some go in the TEST box; 95% right.

It learned from about 2,500 pictures of my hand, taken with the webcam.

During training every picture gets randomly turned, zoomed, mirrored and re-lit, so it can't just memorise them.

Then I test it on recordings it has never seen: it gets 95 percent right, and every count from one to five was perfect.

12. Something new: Grad-CAM

4:01 – 4:25

On screen: Grad-CAM: the heat map on my watch (is 0, says 1, ✗), then three heat maps on fingers (✓).

Here's something we didn't do in class: Grad-CAM.

It uses backpropagation again, but asks a different question: which pixels made you say that?

The answer is a heat map.

And it caught my network cheating.

It had decided my smartwatch meant "one", because I wore it in my "one" pictures.

I recorded more pictures with and without the watch, and now the heat sits on the fingers, where it belongs.

13. On the Arduino 4:25 – 4:42

On screen: The conv blocks hop from the laptop onto the UNO Q, then the safe opens again: 1, 4, 2.

Last, I moved the trained network onto the Arduino UNO Q.

It reads the camera about fifteen times a second, and the network takes about 25 milliseconds per picture.

So: press the button, the lights come on, one…

four…

two…

and it's open.